Kiosky

Funktionsübersicht & Handbuch

Installation · Bedienung · Updates · Betrieb

Kiosky 3.3.5
Home-Assistant-Integration 1.0.0

Stand: 17. Oktober 2026 · Sprache: Deutsch

Ein praktischer Leitfaden für Administratoren, Redaktion und technischen Betrieb. Behandelt die eigenständige Anwendung, das WordPress-Plugin, die TYPO3-Extension und die Anbindung an Home Assistant.

So verwenden Sie dieses Handbuch
Für einen schnellen Einstieg lesen Sie die Funktionsübersicht und „Vom ersten Inhalt zum Display“. Wählen Sie danach die Installation Ihrer Plattform. Vor jeder Aktualisierung sind die Kapitel zu Updates und Datensicherung maßgeblich.

Die Dokumentation basiert auf den drei bereitgestellten Repository-Ständen. Sie beschreibt Kiosky als Produkt; Zugangsdaten, Hostnamen und Verzeichnisse in Beispielen müssen zur eigenen Installation passen.

Inhalt

Kapitel	Thema
01	Produkt und Plattformen
02	Funktionen: Inhalte und Gestaltung
03	Funktionen: Ausspielung und Steuerung
04	Vom ersten Inhalt zum Display
05	Veranstaltungen, Schnittstellen und Werbung
06	Medien, Vorlagen und Inhaltstransfer
07	Standalone installieren
08	Standalone unter Plesk betreiben
09	WordPress installieren
10	TYPO3 installieren
11	Home Assistant über HACS einrichten
12	Externe API und Automationen
13	Updates vorbereiten und Zugänge einrichten
14	Standalone und WordPress aktualisieren
15	TYPO3 und Home Assistant aktualisieren
16	Datensicherung, Wiederherstellung und Betrieb
17	Fehler finden und beheben
18	Technische Referenz und Quellen

Begriffe

Ein **Slide** ist eine einzelne gestaltete Inhaltsseite. Ein **Kanal** ist eine geordnete Playlist aus Slides. Ein **Display** ist ein Ausspielziel; der **Player** zeigt dessen wirksame Inhalte an. **Gruppen** bündeln Displays, **Standorte** ordnen sie räumlich zu. Ein **Zeitplan** legt fest, wann welche Inhalte ausgespielt werden. **Presets** bündeln vorbereitete Steueraktionen.

Darstellung der Beispiele

`DISPLAY_ID`, `CHANNEL_ID`, `GROUP_ID` und ähnliche Begriffe sind Platzhalter. Adressen mit `example.org` sind Beispiele. Geheimnisse werden nicht als feste Werte vorgegeben. Menübezeichnungen können je nach CMS-Version, Sprache und Berechtigung etwas abweichen.

01 · Produkt und Plattformen

Kiosky ist ein webbasiertes Digital-Signage-System für Veranstaltungsorte, Informationsdisplays, Wegweisung und zentral gesteuerte Bildschirme. Die gemeinsame Oberfläche und der Player werden für drei Servervarianten bereitgestellt.

Variante	Laufzeit und Datenhaltung	Geeignet für
Standalone	Node.js/TypeScript, SQLite	Eigenständiger Betrieb ohne CMS
WordPress	PHP, WordPress-Datenbank und Mediathek	Bestehende WordPress-Installationen
TYPO3	PHP, TYPO3-Datenbank, FAL	TYPO3-Installationen mit Backendrechten
Home Assistant	Separate Integration als API-Client	Steuerung einer vorhandenen Kiosky-Instanz

WordPress und TYPO3 benötigen keinen zusätzlichen Kiosky-Standalone-Server. Die Home-Assistant-Integration installiert ihrerseits keinen Kiosky-Server.

Voraussetzungen im Überblick

- **Standalone:** Node.js ab 22.5; empfohlen Node.js 24 LTS. Für Entwicklung und Quellcode-Build pnpm, im geprüften CI-Stand 11.9.0. Für den eingebauten Updater außerdem `unzip`.
- **WordPress:** WordPress ab 6.6. Der Plugin-Header nennt PHP ab 8.1, das allgemeine Distributionsmanifest mindestens 8.2. Für Neuinstallationen konservativ **PHP 8.2 oder neuer** einplanen und CMS-Kompatibilität prüfen.
- **TYPO3:** bevorzugt 13.4 LTS, außerdem TYPO3 14 gemäß Paketanforderungen. Die Kiosky-Extension nennt PHP 8.2–8.4; die gewählte TYPO3-Version kann eine höhere Mindestversion verlangen.
- **Home Assistant:** ab 2026.3.0, HACS und Kiosky ab 3.3.1. Für die Integration selbst ist die Entwicklung mit Python 3.14 dokumentiert.

Welches Paket gehört wohin?

Ausgabe für 3.3.5	Verwendung
`kioski-app-3.3.5.zip`	Standalone-Anwendung
`kioski-wordpress-3.3.5.zip`	WordPress-Plugin
`kioski_3.3.5.zip`	TYPO3-Extension, Classic Mode
`casesound/kiosky`	TYPO3-Installation über Composer

Diese Namen stammen aus dem aktuellen Buildskript und Distributionsmanifest. Die Schreibweise **kioski** in ZIP-Namen ist beabsichtigt. Produktname und Quellcode-Repository heißen **Kiosky** beziehungsweise `kiosky`.

02 · Inhalte und Gestaltung

Slides und visueller Editor

Kiosky unterstützt Text, Bilder, Videos, Webseiten, PDFs und Veranstaltungsinhalte. Im visuellen Editor werden Elemente frei positioniert. Quer- und Hochformat erlauben unterschiedliche Displayausrichtungen. Vorlagen dienen als wiederverwendbarer Ausgangspunkt; Tags und Sichtbarkeitszeiten helfen bei der Organisation.

Für Text lassen sich unter anderem Schriftgröße, Farbe, Hintergrund und automatische Größenanpassung einstellen. Eine Vorschau sollte immer mit der tatsächlichen Player-Darstellung verglichen werden, insbesondere bei langen Texten und abweichenden Bildschirmformaten.

Schriftgrößen und responsive Darstellung

Ab 3.3.4 verwenden die Größenfelder **Punkt (pt)**. Die Entwurfsfläche beträgt 1920 × 1080 im Querformat beziehungsweise 1080 × 1920 im Hochformat. Ein Punkt entspricht auf dieser Entwurfsfläche 96/72 CSS-Pixeln.

Mit **Responsive** wächst die Schrift proportional zur tatsächlichen Breite der Slide. Beispiel: 48 pt entsprechen bei Full HD im Querformat 64 CSS-Pixeln und bei 4K 128 CSS-Pixeln. **Schrift automatisch einpassen** passt den Text zusätzlich an sein Feld an.

Ältere Dokumente ohne Einheitenangabe behalten ihre bisherige Pixeldarstellung. Erst das Bearbeiten der Größe speichert einen Punktwert. Version 3.3.5 verbessert die Bedienbarkeit der Checkboxen in schmalen Eigenschaftenbereichen.

Transparente Lauftexte

1. Slide im Editor öffnen und das Lauftext-Element auswählen.
2. Bei **Element-Hintergrund** die Option **Transparent** wählen.
3. Text, Schriftfarbe und Position einstellen.
4. Speichern und auf dem Player prüfen.

Der Elementhintergrund wird durchsichtig; darunterliegende Inhalte bleiben sichtbar. Die Einstellung macht weder den gesamten Slide noch das Browserfenster transparent.

Kanäle und Playlists

Ein Kanal enthält Slides in festgelegter Reihenfolge. Pro Slide werden Anzeigedauer und gegebenenfalls Übergang festgelegt. Der Kanal bietet einen Standardübergang; einzelne Übergänge können davon abweichen. Per Drag-and-drop lässt sich die Reihenfolge ändern.

Empfohlener Redaktionsablauf
Inhalt erstellen → Darstellung prüfen → Kanal zusammenstellen → Zeitplan oder Displayzuordnung festlegen → Wiedergabe am Ziel kontrollieren. Ein gespeicherter Slide allein garantiert noch keine sichtbare Ausspielung.

03 · Ausspielung und Steuerung

Displays, Gruppen, Standorte und Matrix

Displays besitzen feste Player-Adressen. Gruppen erleichtern die gemeinsame Steuerung mehrerer Bildschirme. Standorte ordnen Inhalte und Ausspielziele räumlich zu. Matrix-Displays unterstützen zusammengehörige Bildschirmflächen.

Der öffentliche Einstieg `/player` bietet das Player Center. Dort stehen Browser-Player und Downloads für Windows, Ubuntu/Debian und macOS bereit. Neue Player können über einen kurzlebigen sechsstelligen Code im Backend einem Display zugeordnet werden. Alternativ wird ein bereits zugeordneter Player am Display heruntergeladen.

macOS-Downloads enthalten eine `.app` im ZIP. Da die Display-Adresse individuell eingebettet wird, ist die App laut Produktdokumentation nicht mit einem Apple-Entwicklerzertifikat signiert. Die dort beschriebene Erstöffnung erfolgt gegebenenfalls über **Rechtsklick → Öffnen**.

Zeitpläne

Die Tages-, Wochen- und Monatsansicht ermöglicht die zeitgesteuerte Zuweisung von Kanälen und einzelnen Slides. Unterstützt werden mehrere Ziele, Prioritäten, Laufzeiten sowie tägliche oder wöchentliche Wiederholungen. Höher priorisierte Ausspielungen werden vor versehentlichem Überschreiben geschützt.

Veranstaltungsabläufe und Display-Zeitpläne sind getrennt: Ein Programmpunkt einer Veranstaltung veröffentlicht nicht automatisch einen Signage-Zeitplan.

Welche Inhalte sieht ein Display?

Die dokumentierte Reihenfolge für externe Steuerung lautet:

1. Gültige direkte externe Zuweisung an das Display.
2. Gültige externe Zuweisung an eine Displaygruppe.
3. Veröffentlichter Zeitplan.
4. Standardkanal des Displays.

Eine passende aktive Sicherheitswarnung übersteuert die normale Darstellung. Eine direkte Display-Zuweisung hat gegenüber einer Gruppenzuweisung Vorrang. Innerhalb derselben Zielebene entscheiden Priorität und anschließend Aktualisierungszeit.

Betriebszentrale und Warnungen

Die Betriebszentrale zeigt Player-Heartbeat, Online-/Offline-Zustand und Cache-Bereitschaft. Synchronisation und Neuladen können gezielt ausgelöst werden. Player nutzen den zuletzt gültigen Inhalt beziehungsweise einen konfigurierten Fallback; fremde Webseiten und externe Medien bleiben von ihrer Erreichbarkeit abhängig.

Warnungen können Gruppen oder einzelne Displays adressieren. Für die External Control API gibt es systemweit eine aktive Emergency-Warnung; eine neue Aktivierung ersetzt die bisherige. Nach dem Ende kehrt der Player zur regulären Ausspielung zurück.

04 · Vom ersten Inhalt zum Display

Einmalige Vorbereitung

1. Die passende Plattform installieren und Kiosky öffnen.
2. In Standalone den ersten Administrator über die Ersteinrichtung anlegen. Im CMS mit einem berechtigten Backendkonto anmelden.
3. Standorte und gegebenenfalls Displaygruppen anlegen.
4. Medienbereitstellung und Benutzerrechte prüfen.

Beispiel: Begrüßungsbildschirm im Foyer

1 · Slide gestalten. Einen Slide „Willkommen im Foyer“ erstellen, Querformat wählen und Text sowie Logo hinzufügen. Ausreichende Schriftgröße und Kontrast verwenden. Speichern und Vorschau prüfen.

2 · Kanal erstellen. Einen Kanal „Foyer Standard“ anlegen. Den Begrüßungsslide und weitere Inhalte hinzufügen. Reihenfolge, Dauer und Übergänge festlegen und den Kanal speichern.

3 · Display anlegen. Ein Display „Foyer links“ mit passender Ausrichtung und Zeitzone erstellen. „Foyer Standard“ als Standardkanal setzen. Bei Bedarf eine Gruppe und einen Standort zuordnen.

4 · Player verbinden. Die im Backend erzeugte Player-Adresse verwenden oder den Player über das Player Center koppeln. Keine Adresse anhand eines vermuteten Slugs zusammensetzen. Auf dem Zielgerät prüfen, ob Inhalt, Format und Auflösung stimmen.

5 · Abweichenden Inhalt planen. Für einen Veranstaltungstag einen eigenen Kanal oder Slide zeitlich zuweisen. Ziel, Datum, Start, Ende, Wiederholung und Priorität kontrollieren. Den Plan veröffentlichen beziehungsweise den vorgesehenen Freigabestatus setzen.

6 · Betrieb prüfen. Im Betriebsbereich den Heartbeat und die Erreichbarkeit prüfen. Kontrollieren, dass nach Ende des Zeitfensters wieder der Standardinhalt erscheint.

Rechte passend vergeben

Standalone besitzt eine eigene Benutzer- und Rollenverwaltung; die Produktübersicht nennt Administration, Redaktion und Betrachter. WordPress und TYPO3 verwenden die jeweiligen CMS-Konten und deren Berechtigungen.

In TYPO3 gibt es zusätzlich die Kiosky-Rollen **Administrator**, **Manager**, **Editor**, **Event editor**, **Display control** und **Read-only**. Modul-, Seiten- und Tabellenrechte der TYPO3-Backendgruppen bleiben erforderlich. Warnaktionen und Einstellungen verlangen passende Berechtigungen.

Abnahme eines neuen Displays

- Richtiger Kanal und richtige Displayzuordnung sichtbar?
- Uhrzeit, Zeitzone und geplante Wechsel korrekt?
- Schrift, Bilder, PDF und Video auf dem tatsächlichen Bildschirm lesbar?
- Player nach Neustart wieder verbunden und Heartbeat aktuell?
- Temporäre externe Zuweisungen und Testwarnungen wieder entfernt?

05 · Veranstaltungen und Werbung

CrewBrain und easyjob verbinden

Beide Quellen können parallel als Veranstaltungsdatenquellen verwendet werden. Verbindung, Zugangsdaten und Feldzuordnung werden im geschützten Schnittstellenbereich eingerichtet. Vor dem ersten Import zuerst den Verbindungstest und die Importvorschau nutzen.

1. Quelle und Basisadresse konfigurieren.
2. Passendes Authentifizierungsverfahren und berechtigten Zugang hinterlegen.
3. Verbindung testen; bei Fehlern zunächst URL, TLS und Berechtigungen prüfen.
4. Feldzuordnung, Zeitraum und Filter in der Vorschau kontrollieren.
5. Einen kleinen manuellen Import durchführen und Ergebnisse prüfen.
6. Erst danach automatische Importe aktivieren und protokollierte Ergebnisse beobachten.

In WordPress übernehmen WordPress-HTTP und WP-Cron die jeweiligen Aufgaben. In TYPO3 können Scheduler-Aufgaben beziehungsweise dokumentierte Konsolenbefehle genutzt werden, zum Beispiel `kiosky:crewbrain:sync`. Ohne eingerichteten Scheduler darf eine regelmäßige Ausführung nicht vorausgesetzt werden.

Veranstaltungsabläufe

Ablaufpunkte enthalten etwa Einlass, Programmbeginn, Pause oder Ende. easyjob-Raumnutzungen können auf solche Ablaufpunkte abgebildet werden. Kategorien, Sichtbarkeit und Bezeichnungen sind konfigurierbar. Interne Punkte werden bei der öffentlichen Ausgabe gefiltert.

Ein Veranstaltungsablauf ist fachliche Programminformation. Die zeitliche Veröffentlichung einer Slide auf einem Bildschirm erfolgt weiterhin über die Displayplanung beziehungsweise Steuerung.

Veranstaltungsbilder aus easyjob

Bildanhänge mit der Dokumentenart **Veranstaltungsbild** werden importiert. Bei Jobs haben deren Anhänge Vorrang; fehlen passende Bilder, werden Projektanhänge verwendet. Mehrere Bilder werden übernommen, das zuletzt geänderte Bild wird primär zugeordnet.

Die Ablage folgt `Werbung / JJJJ / MM / Datum – Veranstaltungstitel`. Unterstützt sind JPEG, PNG, GIF, WebP und AVIF bis 20 MB. Manuell gesperrte Veranstaltungsbilder bleiben erhalten.

Automatische Werbung

Einstellungen → Module → Werbung steuert die Sichtbarkeit. Davon getrennt wird unter **Einstellungen → Werbung** die automatische Erzeugung aktiviert und konfiguriert. Veranstaltungsarten, Standardbilder, Vorlage und Standorte festlegen und speichern.

Bildpriorität: Veranstaltungsbild → Standardbild der Veranstaltungsart → Hintergrund der Werbevorlage. Ohne passende zukünftige Veranstaltungen bleibt der automatische Kanal leer. Beim Deaktivieren werden automatisch erzeugte Slides archiviert und der automatische Kanal pausiert; manuelle Werbeinhalte bleiben erhalten.

06 · Medien und Inhaltstransfer

Wo liegen die Medien?

Plattform	Verwaltung	Im Betrieb beachten
Standalone	Kiosky-Medienbibliothek	Datenverzeichnis und Mediendateien sichern
WordPress	WordPress-Mediathek	Uploads, Anhänge und Datenbank gehören zusammen
TYPO3	FAL und Dateiliste	Öffentlicher, beschreibbarer Speicher; Benutzerfreigaben prüfen

Beim ersten Medienzugriff eines berechtigten Bearbeiters werden frühere Kiosky-CMS-Medien in die native Verwaltung übernommen. Verweise werden angepasst. Bei einem Fehler bleiben Quelldaten für einen erneuten Versuch erhalten. Alte Dateien erst entfernen, wenn Übernahme und Wiedergabe überprüft wurden.

Slides, Kanäle und Vorlagen exportieren

- **Slides:** In „Slides & Kanäle“ auswählen und „Exportieren“ wählen; alternativ im Editor exportieren.
- **Kanäle:** In der Kanalübersicht „Kanal exportieren“ verwenden. Benötigte Slides werden mitgenommen.
- **Vorlagen:** In der Vorlagenverwaltung eine einzelne oder alle Vorlagen exportieren.

Die JSON-Inhaltspakete enthalten den aktuellen Arbeitsstand mit Elementen, Layouts, Tags, Sichtbarkeitszeiten, Reihenfolge, Dauer und Übergängen. Benötigte Veranstaltungs- und Standortinformationen werden ergänzt. Frühere Slide-Versionen gehören nicht zum gezielten Inhaltsexport.

Inhaltspakete importieren

1. „Importieren“ in der passenden Übersicht öffnen.
2. JSON-Paket auswählen.
3. Quelle, Anzahl der Inhalte, Abhängigkeiten und Hinweise prüfen.
4. Mit „Jetzt importieren“ übernehmen.
5. Neue Inhalte, Medien und Zielzuordnung kontrollieren.

Importe erzeugen neue IDs und Kopien. Vorhandene Slides, Kanäle und Vorlagen werden nicht überschrieben. Automatisch erzeugte Werbeinhalte werden als eigenständige Kopien übernommen.

Grenzen und Abhängigkeiten

Lokale benötigte Medien werden eingebettet. Externe Webseiten und externe Medien bleiben Links; beim Export werden fremde Server nicht abgerufen. Pro Datei gilt 20 MB, pro Paket 250 MB. PHP- und Serverlimits können die nutzbare Größe zusätzlich begrenzen.

Benutzer, Geräte, Zugangsdaten und Systemeinstellungen gehören nicht zu einem Inhaltspaket. Für vollständige Übertragungen gibt es einen separaten Systemtransfer. **Ein Inhaltspaket ersetzt kein vollständiges Backup.**

07 · Standalone installieren

Vorbereitung

Node.js 24 LTS, pnpm und einen beschreibbaren Anwendungsordner bereitstellen. Für eine Quellcodeinstallation wird Lesezugriff auf `soerenelvers/kiosky` benötigt; Berechtigungen des eigenen GitHub-Kontos prüfen. Alternativ das passende Standalone-Releasepaket verwenden und dessen enthaltene Installationshinweise beachten.

Die folgenden Befehle beschreiben den dokumentierten **Quellcodeweg**. In Cloud-Aufgaben vorhandene Checkouts wiederverwenden.

```
git clone https://github.com/soerenelvers/kiosky.git
cd kiosky
pnpm install --frozen-lockfile
cp .env.example .env
```

Eine bestehende `.env` nicht überschreiben. Für einen reproduzierbaren Produktionsstand einen geprüften Release-Tag oder Commit auschecken; `main` kann später weitere Änderungen enthalten.

Minimale Konfiguration

In `.env` die Platzhalter aus der Vorlage prüfen und nur tatsächlich verwendete Integrationen aktivieren:

```
KIOSKY_HOST=127.0.0.1
KIOSKY_PORT=8080
KIOSKY_DATABASE_PATH=./data/kiosky.sqlite
KIOSKY_PUBLIC_URL=https://kiosky.example.org
KIOSKY_UPDATE_AUTO_INSTALL=false
KIOSKY_UPDATE_AUTO_RESTART=false
```

Für `KIOSKY_ENCRYPTION_KEY` einen eigenen zufälligen Wert mit mindestens 32 Zeichen setzen. Ein geeigneter Wert lässt sich lokal erzeugen:

```
node -e "console.log(require('node:crypto').randomBytes(32).toString('hex'))"
```

Den Wert geschützt in der Konfiguration speichern und mit sichern. Für den ersten interaktiven Administrator ist `KIOSKY_ADMIN_API_KEY` nicht erforderlich.

Migration, Start und Prüfung

```
pnpm db:migrate
pnpm dev
```

Im Browser lokal Port 8080 öffnen und den ersten Administrator anlegen. Der technische Test lautet `curl --fail http://127.0.0.1:8080/api/health`; erwartet werden `status: ok` und `database: connected`.

Für den dauerhaften Betrieb `pnpm build` ausführen und den gebauten Einstieg `node --env-file-if-exists=.env server.js` aus dem Anwendungsordner über einen Prozessmanager starten. HTTPS über einen passenden Reverse Proxy bereitstellen. Datenverzeichnis, Logs und Neustartverhalten vor Freigabe prüfen.

08 · Standalone unter Plesk

Anwendung einrichten

In Plesk den gewünschten geprüften Quellcode- oder Release-Stand bereitstellen. Für produktive Installationen einen stabilen, nachvollziehbaren Stand wählen; ein Entwicklungsbranch ist kein fester Releasebezug.

Plesk-Einstellung	Wert
Node.js-Version	24 LTS
Paketmanager	npm gemäß Plesk-Anleitung
Anwendungsstamm	Verzeichnis mit `package.json`
Dokumentenstamm	`public` unter dem Anwendungsstamm
Anwendungsstartdatei	`server.js`
Modus während Installation/Build	`development`
Modus nach erfolgreichem Build	`production`

Installation Schritt für Schritt

1. Quellcode bereitstellen und Node.js-Unterstützung aktivieren.
2. Anwendungsmodus vorübergehend auf `development` setzen, damit die Buildwerkzeuge installiert werden.
3. „npm-Installation“ ausführen.
4. Unter „Node.js-Befehle ausführen“ das Skript `build` ausführen.
5. Umgebungsvariablen konfigurieren und Schreibrechte für das Datenverzeichnis prüfen.
6. Auf `production` umstellen und Anwendung neu starten.
7. Gesundheitsprüfung, Oberfläche und Ersteinrichtung testen.

Umgebungsvariablen

Pflicht für die sichere Speicherung externer Zugänge ist ein eigener `KIOSKY_ENCRYPTION_KEY`. Dazu `KIOSKY_DATABASE_PATH=./data/kiosky.sqlite` und die öffentliche HTTPS-Adresse als `KIOSKY_PUBLIC_URL` setzen.

`PORT` **und** `KIOSKY_PORT` **in Plesk nicht manuell setzen.** Plesk stellt `PORT` bereit. Eine aus der lokalen Entwicklung übernommene `.env` darf das nicht mit einem festen Kiosky-Port übersteuern.

Einladungsversand ist optional: `KIOSKY_RESEND_API_KEY` und `KIOSKY_MAIL_FROM` erst mit verifiziertem Absender konfigurieren. CrewBrain- und easyjob-Zugänge nur für tatsächlich verwendete Quellen hinterlegen.

Betrieb und Aktualisierung

Bereitstellung ausführen → gegebenenfalls Abhängigkeiten installieren → `build` ausführen → App neu starten. Vorher die Sicherung aus Kapitel 16 durchführen. Buildwerkzeuge müssen auch beim nächsten Update verfügbar sein; nötigenfalls den Installationsmodus erneut auf `development` setzen.

Plesk erfasst stdout/stderr. Die Anwendung schreibt strukturierte Einträge unter anderem für Serverstart, Importfehler, langsame Requests und Prozessfehler. Bei Problemen nach `event`, `level` und `requestId` suchen.

09 · WordPress installieren

Voraussetzungen und Paket

Eine unterstützte WordPress-Installation mit passender PHP-Version und Administratorzugang vorbereiten. Für einen neuen Betrieb empfiehlt dieses Handbuch mindestens PHP 8.2; die gewählte WordPress-Version und das Hosting müssen dazu passen.

Das Paket `kioski-wordpress-3.3.5.zip` aus der freigegebenen Distribution beziehen. Kein TYPO3-ZIP und kein GitHub-Quellcodearchiv als Plugin hochladen. Das Plugin-Release besitzt den obersten Ordner `kioski/`.

Erstinstallation

1. Als WordPress-Administrator anmelden.
2. **Plugins → Installieren → Plugin hochladen** öffnen.
3. WordPress-Release-ZIP auswählen und installieren.
4. Kiosky aktivieren.
5. Kiosky im WordPress-Administrationsmenü öffnen.
6. Standorte, Gruppen, erste Inhalte und ein Display anlegen.
7. Mediathek, Player-Adresse und externe API bei Bedarf prüfen.

Kiosky läuft innerhalb von WordPress. Weder ein Node.js-Dienst noch eine zusätzliche Standalone-Instanz sind nötig. Benutzeranmeldung und Medienverwaltung erfolgen über WordPress.

Medien und Rechte

Vorhandene WordPress-Anhänge stehen in der Medienauswahl bereit. Neue Uploads werden als WordPress-Anhänge angelegt. Die interne Standalone-Medien- und Benutzerverwaltung wird im WordPress-Kontext nicht zusätzlich betrieben.

Vor einer Übernahme älterer Kiosky-Medien die Datenbank und insbesondere `wp-content/uploads` sichern. Mit einem ausreichend berechtigten Konto die Medienverwaltung öffnen und die Übernahme anschließend an realen Slides prüfen.

Automatische Aufgaben und Player

Automatische Importe verwenden WP-Cron. Auf selten besuchten Websites können geplante Aufgaben verspätet laufen; für verlässliche Ausführung einen zur Hostingumgebung passenden serverseitigen Cron einrichten.

Das Player Center liegt unter `/player`; bestehende `/kiosky-player/`-Adressen bleiben unterstützt. Die im Kiosky-Backend erzeugte Adresse ist maßgeblich, insbesondere bei Unterverzeichnisinstallationen.

Externe Steuerung vorbereiten

In **Schnittstellen → Externe API → API-Zugänge** einen eigenen Maschinenzugang anlegen. Die typische Basisadresse lautet `/wp-json/kiosky/v1/control`. Ohne sprechende Permalinks ist ein Einstieg über `?rest_route=/kiosky/v1/control` möglich. Home Assistant verarbeitet diese Form automatisch.

Der Webserver muss den Authorization-Header an WordPress weiterreichen. Für Maschinenzugänge ist kein angemeldeter CMS-Browser erforderlich.

10 · TYPO3 installieren

TYPO3 13.4 LTS ist die bevorzugte Zielversion; TYPO3 14 wird gemäß Paketanforderungen ebenfalls unterstützt. PHP-Version und Erweiterungen müssen sowohl zu TYPO3 als auch zu Kiosky passen. Kiosky läuft eigenständig innerhalb von TYPO3 ohne Node.js-Server.

Variante A · Composer

Im **TYPO3-Projektverzeichnis** ausführen:

```
composer config repositories.kiosky vcs \
  https://github.com/soerenelvers/kiosky-distribution.git
composer require casesound/kiosky
vendor/bin/typo3 extension:setup --extension=kiosky
vendor/bin/typo3 cache:flush
```

Bei geschützter Distribution den benötigten lesenden GitHub-Zugang vorher über Composers geschützte `auth.json` oder `COMPOSER_AUTH` bereitstellen. Ein Token gehört nicht in das Projekt-Repository. Öffentliche Distributionen können abhängig vom Zugriff ohne separaten Repository-Token lesbar sein.

Composer-Versionsgrenzen und `composer.lock` werden Teil des eigenen TYPO3-Deployments. Vor Änderungen an einer bestehenden Installation sichern und geplante Abhängigkeitsänderungen prüfen.

Variante B · Classic Mode

1. Das Releasepaket `kioski_3.3.5.zip` bereitstellen.
2. Im TYPO3-Backend **System → Extensions → Upload Extension** öffnen.
3. Das Paket installieren und die Extension aktivieren.
4. Gegebenenfalls Datenbankschema über die vorgesehenen TYPO3-Werkzeuge aktualisieren und Caches leeren.
5. Das Kiosky-Backendmodul öffnen.

Datensätze und Berechtigungen

Eine Speicherseite für Kiosky-Datensätze anlegen beziehungsweise konfigurieren. Backendgruppen benötigen Zugriff auf das Kiosky-Modul, die Speicherseite und erforderliche Tabellen. Am Backendbenutzer die passende **Kiosky role** vergeben. TYPO3-Administratoren besitzen vollen Zugriff.

Einen öffentlichen, beschreibbaren FAL-Speicher wählen. `fileadmin/kiosky/` ist ein möglicher, aber kein vorgeschriebener Pfad. Dateifreigaben und Benutzerrechte müssen zum Speicher passen.

Player und wiederkehrende Aufgaben

Ein Display erstellen und dessen generierte Player-Adresse verwenden. Die TYPO3-Dokumentation nennt die kanonische Route `/kiosky/display/{uuid}`. Bei Bedarf einen Token über `vendor/bin/typo3 kiosky:display:token DISPLAY_UID` erzeugen; den ausgegebenen Klartext geschützt behandeln.

Für automatische Importe den Scheduler einrichten. Zusätzlich können Updateprüfung und `kiosky:housekeeping` geplant werden. Vor produktiver Freigabe Medien, öffentliche Player-Route und tatsächliche Ausführung der Aufgaben prüfen.

11 · Home Assistant über HACS

Voraussetzungen

Home Assistant ab 2026.3.0, HACS und eine aus dem Home-Assistant-Netz erreichbare Kiosky-Instanz ab 3.3.1. Die eigenständige Integration hat im geprüften Stand Version 1.0.0.

In Kiosky einen eigenen Maschinenzugang unter **Schnittstellen → Externe API → API-Zugänge** erstellen. Den einmal angezeigten Schlüssel sicher speichern und die vollständig angezeigte API-Basisadresse übernehmen.

Installation

1. In HACS das Menü ⋮ → **Benutzerdefinierte Repositories** öffnen.
2. `https://github.com/soerenelvers/hacs-kiosky` eintragen.
3. Den Typ **Integration** wählen und hinzufügen.
4. Nach **Kiosky** suchen und herunterladen.
5. **Home Assistant neu starten.**
6. **Einstellungen → Geräte & Dienste → Integration hinzufügen → Kiosky** öffnen.
7. Name, externe API-Basisadresse und Kiosky-API-Schlüssel eingeben.
8. Verbindungstest und angelegte Entitäten prüfen.

Keinen GitHub-Update-Token in das API-Schlüsselfeld eintragen. Eine Player- oder Backendadresse ist ebenfalls keine API-Basisadresse. Installationsunterverzeichnisse müssen erhalten bleiben. Weiterleitungen werden nicht verfolgt; deshalb die endgültige Adresse verwenden.

Bereitgestellte Entitäten

- Kanalauswahl für Displays und Displaygruppen.
- Schaltflächen für gespeicherte Presets.
- Binärsensor für die aktive Emergency-Warnung.
- Diagnosesensoren mit Ressourcen und IDs für Automationen.

Die Kanalauswahl zeigt die **direkte externe Zuweisung**, nicht den gesamten wirksamen Playerzustand. `[Kiosky schedule]` entfernt diese direkte Zuweisung. Eine Gruppenzuweisung kann danach weiterhin gelten; Warnungen bleiben erhalten. Bei direkter Slide-Zuweisung ist der Kanalzustand unbekannt.

Intervall und mehrere Installationen

Standardmäßig wird alle **30 Sekunden** abgefragt. Unter **Konfigurieren** ist ein Intervall von 10–3600 Sekunden möglich. Steuerbefehle lösen eine direkte Statusabfrage aus. Bei mehreren Installationen in Aktionen die passende `config_entry_id` auswählen.

Ungültige Schlüssel lösen eine erneute Authentifizierung aus. Über **Neu konfigurieren** lässt sich ein neuer Schlüssel eintragen. Eine andere API-Adresse als neue Installation hinzufügen. Nach HACS-Updates Home Assistant neu starten.

12 · Externe API und Automationen

Adressen und Verbindungstest

Plattform	Beispiel für die Basisadresse
Standalone	`https://kiosky.example.org/api/v1`
WordPress	`https://example.org/wp-json/kiosky/v1/control`
TYPO3	`https://example.org/kiosky-api/v1`

Im Zweifel die in Kiosky angezeigte Adresse verwenden. Ein eigener Kiosky-API-Benutzer ist auf die External Control API begrenzt und kann keine Benutzerverwaltung, Medienverwaltung oder Updates bedienen.

```
# KIOSKY_API_KEY zuvor sicher bereitstellen.
export KIOSKY_API_BASE='https://kiosky.example.org/api/v1'
curl --fail-with-body "$KIOSKY_API_BASE/displays" \
  --header "Authorization: Bearer $KIOSKY_API_KEY"
```

Alternativ wird `X-API-Key` unterstützt. Ressourcen-IDs über die API oder die Home-Assistant-Diagnosesensoren ermitteln. Bei WordPress mit `?rest_route=` den Ressourcenpfad an den Wert dieses Parameters anhängen, nicht hinter eine beliebige Query-URL.

Beispiel · Kanal in Home Assistant zuweisen

```
action: kiosky.assign_content
data:
  target_type: display
  target_id: DISPLAY_ID
  content_type: channel
  content_id: CHANNEL_ID
  priority: 200
```

Beispiel · Informationswarnung für eine Gruppe

```
action: kiosky.activate_emergency
data:
  warning_type: information
  title: "Veranstaltung beendet"
  instructions: "Bitte nutzen Sie den Hauptausgang."
  targets:
    - type: group
      id: GROUP_ID
```

Weitere Aktionen sind `clear_assignments`, `execute_preset`, `end_emergency` und `refresh`. `clear_assignments` **beendet keine Warnung.** `end_emergency` beendet die systemweit aktive Emergency-Warnung. Presets ohne gespeicherte Ziele benötigen explizite Ziele in der Aktion.

API-Details und vollständige Felder stehen in `docs/external-api.md` und `docs/openapi.yaml`. Produktive Integrationen über HTTPS betreiben; Zugangsschlüssel nicht in URLs oder Browsercode einbauen.

13 · Updates vorbereiten

Vor jedem Update

1. Installierte Version, Plattform und gewünschte Zielversion notieren.
2. Releasehinweise und Kompatibilität mit Node.js, PHP und CMS lesen.
3. Datenbank, Dateien, Medien und geschützte Konfiguration konsistent sichern.
4. Wiederherstellbarkeit prüfen; für wichtige Installationen zuerst auf einer Testkopie aktualisieren.
5. Updateweg und Wartungsfenster festlegen.
6. Nach dem Update Version, Anmeldung, Player, Medien und geplante Aufgaben prüfen.

Die Version eines Updates sollte anhand des aktuellen freigegebenen Stable-Feeds beziehungsweise Release-Tags gewählt werden. **3.3.5 ist der Stand dieses Handbuchs, keine dauerhafte Aussage über die neueste Version.**

Zwei Zugänge mit unterschiedlichem Zweck

Zugang	Herkunft	Verwendung
GitHub-Update-Token oder Update-Lizenz	Distribution beziehungsweise Updateanbieter	Metadaten und Pakete laden
Kiosky-API-Schlüssel	Kiosky → Externe API → API-Zugänge	Inhalte, Displays, Presets und Warnungen steuern

Ein Kiosky-API-Schlüssel ersetzt keinen Distributionszugang. Home Assistant benötigt den Kiosky-Schlüssel, nicht den Update-Token. Das öffentliche HACS-Repository nutzt die reguläre GitHub-Anbindung von HACS.

GitHub-Zugang einrichten

Für einen erforderlichen GitHub-Zugang einen **Fine-grained personal access token** mit minimalen Leserechten verwenden. Wenn das Konto den passenden Resource owner auswählen darf: nur `kiosky-distribution` freigeben und **Contents: Read-only** setzen; Metadata bleibt lesbar.

Bei öffentlicher Distribution kann ein eigener lesender Zugang für öffentliche Repositories genügen. Bei privater Distribution muss das Konto bereits Zugriff besitzen. Ein Token verleiht keine zusätzlichen Kontorechte; ein fremdes persönliches Konto kann nicht beliebig als Resource owner ausgewählt werden.

Den Token geschützt speichern, Ablauf und Widerruf berücksichtigen. WordPress verwendet **Einstellungen → Kiosky Updates**; TYPO3 nutzt Composer-Authentifizierung und für den Backendweg gegebenenfalls `updateAccessToken`; Standalone nutzt `KIOSKY_UPDATE_ACCESS_TOKEN`.

Paketprüfung

Releasepakete besitzen SHA-256-Prüfsummen im Manifest beziehungsweise in `SHA256SUMS.txt`. Die Updater prüfen Kompatibilität und Paketintegrität. Bei einer Abweichung die Ursache klären und das Paket neu aus der vertrauenswürdigen Distribution beziehen. Prüfsummenprüfung und TLS nicht abschalten.

14 · Standalone und WordPress aktualisieren

Standalone · eingebauter Updater

Im Backend **Updates** öffnen. Für den verwendeten Distributionszugang die passende HTTPS-Manifestadresse und den benötigten Zugang konfigurieren:

```
KIOSKY_UPDATE_MANIFEST_URL=https://updates.example.org/api/v1/standalone/latest
KIOSKY_UPDATE_ACCESS_TOKEN=ZUGANG_IN_GESCHUETZTER_KONFIGURATION
KIOSKY_UPDATE_AUTO_CHECK=true
KIOSKY_UPDATE_AUTO_INSTALL=false
KIOSKY_UPDATE_AUTO_RESTART=false
KIOSKY_UPDATE_CHECK_INTERVAL_HOURS=24
```

Die Adresse ist ein Beispiel für einen eigenen Update-Dienst; den tatsächlich bereitgestellten Feed verwenden. Standardmäßig wird täglich geprüft. Manuell nach Updates suchen, Releasehinweise lesen und die Installation auslösen. Der Updater validiert das Manifest und SHA-256, verwirft unsichere Archivpfade und sichert verwaltete Anwendungsdateien. `unzip` muss verfügbar sein.

Automatische Installation und Neustart nur mit eingerichtetem Supervisor aktivieren, etwa systemd oder einer geeigneten Container-Neustartregel. `.env` und Datenbank werden nicht ersetzt, müssen aber unabhängig gesichert sein.

Standalone · Quellcode oder Plesk

Nach Sicherung den gewünschten geprüften Stand bereitstellen, zum Lockfile passende Abhängigkeiten installieren, den Build ausführen, Migrationen anwenden und die Anwendung neu starten. Bei pnpm-Quellcodeinstallationen sind die Kernschritte:

```
pnpm install --frozen-lockfile
pnpm build
pnpm db:migrate
```

Die Befehle aus dem Anwendungsordner im vorgesehenen Wartungsablauf ausführen. In Plesk dessen Bereitstellung, npm-Installation, Build und Neustart verwenden. Nicht mehrere Deployment- und Updatewege gleichzeitig auf dieselbe Installation anwenden.

WordPress

1. Unter **Einstellungen → Kiosky Updates** erforderlichen Zugang speichern.
2. **Kiosky → Updates → Nach Updates suchen** wählen.
3. Zielversion und Hinweise prüfen, dann installieren.
4. Alternativ **Dashboard → Aktualisierungen → Erneut prüfen** und Pluginupdate verwenden.
5. Version, Player und native Medien prüfen.

Ohne funktionierenden Updater das neue WordPress-Release-ZIP hochladen und das bestehende Plugin ersetzen. **Vorher nicht deinstallieren.** WordPress plant Prüfungen ungefähr zweimal täglich; Kiosky cached erfolgreiche Manifestabfragen sechs Stunden. Das ist kein eigener Sechs-Stunden-Zeitplan. Automatische Plugininstallation wird getrennt in WordPress aktiviert.

15 · TYPO3 und Home Assistant aktualisieren

TYPO3 · Composer

Im Projektverzeichnis nach der Sicherung zunächst die erwartete Aktualisierung und Abhängigkeiten prüfen. Der dokumentierte manuelle Ablauf lautet:

```
composer outdated casesound/kiosky
composer update casesound/kiosky --with-all-dependencies
vendor/bin/typo3 extension:setup --extension=kiosky
vendor/bin/typo3 cache:flush
```

Der erlaubte Zielbereich ergibt sich aus `composer.json`. Vorher einen Composer-Dry-Run zur Beurteilung zusätzlicher Änderungen verwenden. Nicht ungeprüft weitere CMS-Abhängigkeiten aktualisieren.

Der Kiosky-Backendweg bietet die Installation erst an, wenn Composer genau die Stable-Version auflösen kann. Der Webserverbenutzer benötigt Zugriff auf Composer und auf die geschützte Authentifizierung; ein globaler Composer-Zugang des SSH-Benutzers reicht dafür nicht automatisch aus.

Der Backend-Updater sichert Composer-Dateien und versucht bei Fehlern deren Wiederherstellung einschließlich Vendor-Stand. **Datenbank und FAL sind weiterhin Verantwortung des Betreibers.** Eine Rücknahme von Composer-Dateien macht Datenmigrationen nicht automatisch rückgängig.

TYPO3 · Classic Mode

1. Backup einschließlich Datenbank und FAL erstellen.
2. Das angebotene `kioski_VERSION.zip` im Extension Manager hochladen und die bestehende Extension aktualisieren.
3. Erforderliche Schemaaktualisierung ausführen und Caches leeren.
4. Backend, Player, Medien und Version prüfen.

Classic-Installationen können Updates im Backend prüfen. Die laufende Extension wird dort nicht durch einen beliebigen PHP-Webrequest überschrieben; das Paket wird über den vorgesehenen Extension-Installationsweg eingespielt.

Beim Öffnen des Kiosky-Backends werden fehlende oder mehr als 24 Stunden alte Prüfungen erneuert. Optional den Scheduler-Befehl `kiosky:update:check` täglich ausführen lassen.

Home Assistant / HACS

1. Home-Assistant-Sicherung anlegen.
2. Verfügbare Aktualisierung der Kiosky-Integration in HACS öffnen.
3. Hinweise und benötigte Home-Assistant-Version prüfen.
4. Update herunterladen und **Home Assistant neu starten**.
5. Integration, Entitäten und eine kontrollierte Steueraktion prüfen.

Server und Integration besitzen unabhängige Versionsnummern. Bei Schlüsselwechsel die erneute Authentifizierung beziehungsweise **Neu konfigurieren** nutzen. Die API-Adresse bei einem Serverumzug als neue Installation einrichten.

16 · Sicherung und laufender Betrieb

Was gehört in ein vollständiges Backup?

Plattform	Zu sichernde Bestandteile
Standalone	SQLite-Datenbank, Medien/Datenverzeichnis, `.env` beziehungsweise sichere Variablenablage, Verschlüsselungsschlüssel, eingesetzter Programmstand
WordPress	WordPress-Datenbank, `wp-content/uploads`, relevante Konfiguration, Pluginstand und Deploymentinformationen
TYPO3	Datenbank, FAL-Dateispeicher, Projektkonfiguration, `composer.json`, `composer.lock` und Wiederherstellungsweg des Codes
Home Assistant	Home-Assistant-Backup, Konfiguration und Integrationsstand

Zugangsdaten und Schlüssel im Backup verschlüsselt beziehungsweise angemessen geschützt aufbewahren. **Ohne den passenden Verschlüsselungsschlüssel können gespeicherte externe Zugangsdaten unbrauchbar werden.**

Konsistent sichern

Für SQLite entweder den Dienst für eine konsistente Sicherung stoppen oder ein geeignetes SQLite-Backupverfahren verwenden. Eine laufend geschriebene Datenbank nicht isoliert kopieren und vorhandene WAL-Dateien ignorieren. Bei CMS-Systemen Datenbank und zugehörige Dateien auf einen zusammenpassenden Stand bringen.

Sicherungen außerhalb des Anwendungsordners ablegen, Zugriff begrenzen und Aufbewahrungsfristen festlegen. Wiederherstellung regelmäßig auf einer getrennten Testinstanz prüfen.

Wiederherstellung und Rückkehr zu einer älteren Version

1. Betroffenen Dienst in den vorgesehenen Wartungszustand bringen.
2. Passenden Programmstand wiederherstellen.
3. Bei nicht rückwärtskompatiblen Migrationen die **dazugehörige Datenbanksicherung** einspielen.
4. Medien, Konfiguration und Schlüssel aus demselben Sicherungsstand wiederherstellen.
5. Im Composer-Projekt die gesicherte Lockdatei mit `composer install` anwenden.
6. Caches über den vorgesehenen Plattformweg leeren.
7. Anmeldung, Daten, Player, externe API und automatische Aufgaben testen.

Ein älteres ZIP allein ist kein verlässlicher Datenbank-Rollback. Produktive Daten nicht durch einen ungeprüften Downgrade gefährden.

Regelmäßige Betriebsprüfung

Heartbeat und Playerzustand, freier Speicher, Backup-Erfolg, Ablauf von Tokens und Zertifikaten sowie Import- und Fehlerlogs prüfen. Nach Änderungen an Zeitplänen Zeitzone und Sommerzeit berücksichtigen. Einen eigenen widerrufbaren API-Zugang pro Integration verwenden.

Der Inhaltsexport aus Kapitel 06 ist für gezielte Übertragungen gedacht und ersetzt diese Sicherung nicht.

17 · Fehler finden und beheben

Symptom	Zuerst prüfen
Standalone startet nicht	Node-Version, Build vorhanden, Schreibrechte für Daten, Log, Portkonflikt
Plesk zeigt Fehler	`server.js`, Dokumentenstamm `public`, Build erfolgreich, kein eigener `PORT`/`KIOSKY_PORT`
TYPO3-Modul fehlt	Extension aktiviert, Modul- und Gruppenrechte, Caches
TYPO3-Listen bleiben leer	Speicherseite, Tabellenrechte, Kiosky-Rolle
CMS-Medien fehlen	Uploads/FAL vorhanden, Dateispeicher öffentlich und beschreibbar, Benutzerfreigaben
Player offline	Netzwerk, richtige Adresse, HTTPS, Geräteuhr, Heartbeat und Token
Falscher Inhalt sichtbar	Direkte Zuweisung, Gruppe, Zeitplan, Standardkanal und aktive Warnung in dieser Reihenfolge
Keine automatische Werbung	Modul sichtbar, Generator separat aktiviert, passende zukünftige Veranstaltungen, Sync
Kein automatischer Import	Zugang und Vorschau, WP-Cron beziehungsweise TYPO3-Scheduler tatsächlich aktiv
Update wird nicht angeboten	Stable-Version, Kompatibilität, Token, Cache und bei TYPO3 auflösbarer Composer-Tag

API- und Home-Assistant-Fehler

- **401:** Schlüssel fehlt, ist ungültig oder widerrufen. Kiosky-Schlüssel statt Update-Token verwenden.
- **403:** Berechtigungen und verwendeten Endpunkt prüfen.
- **404:** Vollständige API-Basisadresse, Unterverzeichnis und Ressourcenpfad prüfen.
- **400/422:** Datenformat, IDs, Zeitfenster und Priorität kontrollieren.
- **TLS-/Verbindungsfehler:** Erreichbarkeit aus dem tatsächlichen Home-Assistant-Netz und Zertifikatskette prüfen.
- **Weiterleitung:** Die endgültige API-Adresse eintragen; die Integration verfolgt Redirects nicht.

Bei WordPress zusätzlich die Weitergabe des Authorization-Headers prüfen. Eine im Browser funktionierende CMS-Anmeldung belegt noch keinen funktionierenden Maschinenzugang.

Wenn eine Zuweisung scheinbar bestehen bleibt

`clear_assignments` entfernt die angegebenen externen Zuweisungen, beendet aber keine aktive Warnung. Eine Anzeige kann weiterhin durch eine Gruppenzuweisung, einen Zeitplan oder den Standardkanal entstehen. Eine Emergency-Warnung mit der passenden Aktion separat beenden.

Angaben für eine Fehleranalyse

Plattform, Version, Uhrzeit, betroffene Aktion, HTTP-Status und gegebenenfalls `X-Request-ID` notieren. Relevante Logs gezielt bereitstellen und vorher Tokens, Passwörter, persönliche Daten und geschützte Player-Adressen entfernen.

18 · Referenz und Quellen

Häufig benötigte Einstellungen

Einstellung	Zweck
`KIOSKY_DATABASE_PATH`	SQLite-Datei in Standalone
`KIOSKY_ENCRYPTION_KEY`	Verschlüsselung gespeicherter Zugangsdaten
`KIOSKY_PUBLIC_URL`	Öffentliche Adresse, etwa für Einladungen
`KIOSKY_HOST` / `KIOSKY_PORT`	Lokaler Listener; Plesk-Portregel beachten
`KIOSKY_UPDATE_MANIFEST_URL`	Verwendeter Update-Feed
`KIOSKY_UPDATE_ACCESS_TOKEN`	Geschützter Distributionszugang
`KIOSKY_UPDATE_AUTO_*`	Prüfung, Installation und Neustart getrennt
`KIOSKY_RESEND_API_KEY` / `KIOSKY_MAIL_FROM`	Optionaler Einladungsversand

Quellenbasis dieses Handbuchs

- Kiosky-Quellcode, Commit `6a6ba7fd18e499ce2e09ba9d5631b0fe20bb2f86`.
- Kiosky-Distribution, Commit `66c43a590f7594e5b83108e480664cae88e76a99`.
- Home-Assistant-Integration, Commit `f67cd3441edd696d04e8f5275c9d9d9aea363bc4`.

Verwendet wurden insbesondere README, `docs/setup-guide.md`, `docs/development.md`, `docs/plesk.md`, `docs/content-packages.md`, `docs/external-api.md`, `docs/wordpress-updates.md`, `docs/typo3-updates.md`, die TYPO3-Dateien in `Documentation/`, das Distributionsmanifest und die aktuellen Paketmetadaten. Bei älteren widersprüchlichen Dateinamen wurden Buildskript und Manifest für 3.3.5 herangezogen.

Öffentlich ergänzend verfügbar: Kiosky-Einrichtungsanleitung im HACS-Repository und Automationsaktionen.

In der Cloud-Entwicklungsumgebung geprüft

159 Kiosky-Tests, sechs PHP-Vertragstests und 22 Home-Assistant-Tests einschließlich echter Server-/Playersteuerung bestanden. Standalone-, WordPress- und TYPO3-Pakete wurden gebaut. Datenbankmigrationen, Server-Healthcheck, Oberfläche und Player Center wurden geprüft; die PHP-Dateien der Distribution bestanden die Syntaxprüfung.

Geltungsbereich: Diese Prüfungen belegen den eingerichteten Entwicklungsablauf. Vollständige produktive CMS-Installationen und externe Live-Dienste wurden in dieser Cloud-Einrichtung nicht getestet. Die Installations- und Updateanleitungen sind aus den Repositoryquellen abgeleitet und müssen auf die Zielumgebung angewendet werden.

Dokument erstellt am 17. Oktober 2026. Kiosky: Copyright © 2026 Sören Elvers; Lizenzbedingungen gemäß jeweiliger Distribution. Die eigenständige Home-Assistant-Integration wird im Repository unter MIT-Lizenz geführt.